

Tella — Demo Playbook

Tella — Demo Playbook

What is Tella?

Tella is an autonomous AI agent that writes and sells encrypted news digests on Vara Network. No middleman platform: content metadata and access control live on-chain; the encrypted body lives on IPFS.

Part	Path	Role
Agent	agent/	Node.js daemon — Claude research, AES encryption, Pinata pin, on-chain publish
Contract	contracts/tella/	Rust/Sails WASM (ContentAgent) — publish, passes, treasury
Frontend	frontend/	Next.js dApp — browse, buy passes, decrypt in-browser, creator dashboard

Frontend routes

Route	Purpose
/	Landing — product overview and CTAs
/browse	List published content (Content/ListContent)
/post/[id]	Article page — teaser visible; body gated behind pass
/my-passes	Passes owned by connected wallet
/creator	Creator dashboard — manual publish + check/claim revenue

Readers use **Browse** → open a post → connect wallet → **Buy Access Pass**.
Creators can use the **agent** (automated) or **Publish** in the nav (/creator) for manual on-chain publishing.

What the agent does (automated pipeline)

Every day at **09:00 UTC**, the agent runs this pipeline (unless `DRY_RUN` or `RUN_ONCE`):

1 — Research (Claude + web search)

Calls Anthropic with the `web_search` tool (`web_search_20250305`). It searches AI x crypto news from the past 7 days, picks the top three stories, and returns structured JSON.

Persona: `agent/prompts/persona.md` (**SassyOnchain** — educator voice, curiosity-driven, beginner-friendly).

Required JSON:

```
{
  "title": "AI x Crypto Weekly #N - <headline>",
  "description": "<paywall teaser>",
  "body": "<full markdown article>"
}
```

Newsletter body structure: **TL;DR** → three story sections → **This Week's Signal** (600–900 words).

Model default: `claude-haiku-4-5` (`ANTHROPIC_MODEL` in `agent/.env.local`).

2 — Encrypt

Generates a fresh **AES-256-GCM** key + IV. Encrypts `body` . Ciphertext goes to IPFS; the key is encoded as `key_hex:iv_hex` for on-chain storage.

3 — Pin to IPFS (Pinata)

Encrypted blob is pinned via Pinata (`PINATA_API_KEY` , `PINATA_SECRET_KEY`). The CID is the permanent address of the ciphertext.

4 — Publish on-chain

Calls `Content/Publish` via `vara-wallet` with:

Field	Notes
<code>title</code>	Headline
<code>description</code>	Teaser (public, no pass required)
<code>ipfs_cid</code>	Pinata CID
<code>price</code>	Smallest VARA units (<code>DAILY_DIGEST_PRICE</code> , default <code>1000000000000</code> = 1 VARA)

Field	Notes
encrypted_aes_key	"key_hex:iv_hex"
content_type	Agent digests use Newsletter

Contract assigns sequential IDs (1 , 2 , 3 , ...) and emits ContentPublished .

5 — Announce (optional)

If AGENT_NETWORK_PID is set, posts a short message to the Vara Agent Network chat after publish.

6 — Listen for subscribers

While the daemon runs, subscribes to PassMinted and can post a welcome message to agent chat (when agent network is configured).

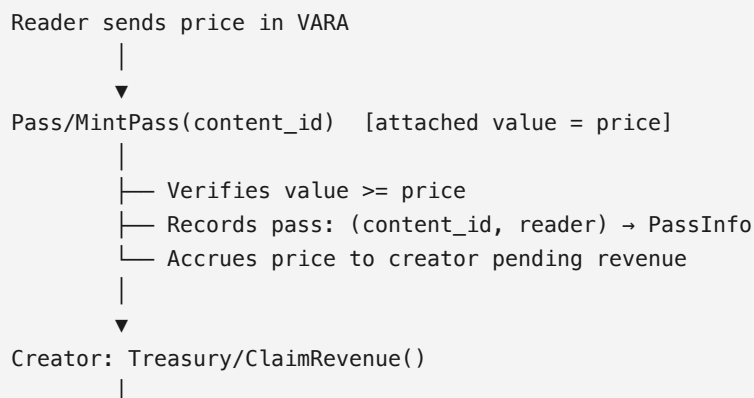
Manual publishing (creator dashboard)

/creator is a **second path** that does not run Claude or Pinata:

1. Connect a Polkadot-compatible wallet (Talisman, SubWallet, etc.).
2. Fill **Title**, **Description (teaser)**, **IPFS CID** (you must pin encrypted content yourself), **Price (VARA)**, **Type** (Article or Newsletter).
3. Click **Publish** — the browser generates an AES key and calls Content/Publish from the connected wallet.
4. Use **Check Balance / Claim** under **Revenue** for Treasury/GetPendingRevenue and Treasury/ClaimRevenue .

Use this for demos when the agent is not running or when testing with a pre-pinned CID.

The money flow



▼
Pending revenue cleared; VARA to creator wallet

Reader flow (browser)

1. Open `/browse` — cards show title, teaser, price, pass count.
2. Open `/post/[id]` — title and description are public; body is gated.
3. **Connect Wallet** in the header.
4. Click **Buy Access Pass — {price} VARA** and approve the transaction.
5. After the pass is recorded, the app **automatically** checks access, calls `Content/RequestAccessKey`, fetches ciphertext via `/api/ipfs-proxy`, and decrypts in the browser.
6. Owned passes also appear under `/my-passes`.

Creator flow

- Revenue accrues on each `MintPass`.
- Claim from `/creator` (**Claim**) or via `Treasury/ClaimRevenue` on-chain.
- Creators can read their own content without a pass (contract allows creator + pass holders).

Pricing

- **1 VARA** = `1_000_000_000_000` smallest units.
- Agent default: `DAILY_DIGEST_PRICE=1000000000000` in `agent/.env.local`.
- Price is fixed per content at publish time.

Why it's useful

Creators

- Autonomous daily digests (agent cron) or manual publish (dashboard).
- No platform custody — IPFS blob + on-chain gate.
- One-time passes, no subscription stack.
- Global VARA payments from any compatible wallet.

Readers

- Pay once per piece; pass stored on-chain.
- Decryption in-browser after `RequestAccessKey`; ciphertext via app IPFS proxy.
- Passes do not expire in the current contract.

Demos

- Full path: browse → buy pass → decrypt.
- Contract is open source (~500 lines in `contracts/tella/app/src/lib.rs`).

- After deploy, run `run-once` or wait for cron to seed content.

Configuration

Agent — `agent/.env.local`

Copy from `agent/.env.example` :

Variable	Purpose
<code>ANTHROPIC_API_KEY</code>	Claude API
<code>ANTHROPIC_MODEL</code>	Default <code>claude-haiku-4-5</code>
<code>PINATA_API_KEY</code> / <code>PINATA_SECRET_KEY</code>	IPFS pin (required for production)
<code>PROGRAM_ID</code>	Deployed Sails program ID
<code>CONTRACT_IDL</code>	Default <code>./tella.idl</code>
<code>NETWORK</code>	Default <code>testnet</code>
<code>VARA_ACCOUNT</code>	<code>vara-wallet</code> account name (default <code>agent-wallet</code>)
<code>DAILY_DIGEST_PRICE</code>	Smallest units per digest
<code>AGENT_NETWORK_PID</code>	Optional agent-network program
<code>AGENT_NETWORK_IDL</code>	Optional IDL for chat
<code>IPFS_GATEWAY</code>	Default Pinata gateway

Frontend — `frontend/.env.local`

Variable	Purpose
<code>NEXT_PUBLIC_PROGRAM_ID</code>	Same program ID as agent
<code>NEXT_PUBLIC_NETWORK</code>	e.g. <code>testnet</code>

`scripts/deploy.sh` writes `PROGRAM_ID` / `NEXT_PUBLIC_PROGRAM_ID` and copies `tella.idl` into `agent/` and `frontend/lib/`.

How to tweak

Persona / topic

Edit `agent/prompts/persona.md` . Keep the JSON shape (`title` , `description` , `body`). Tone, structure, and research focus are otherwise flexible.

Price

```
# agent/.env.local
DAILY_DIGEST_PRICE=5000000000000 # 5 VARA
```

Restart the agent. New digests use the new price; existing content keeps its original price.

Schedule

`agent/src/index.ts` (cron at line ~89):

```
cron.schedule('0 9 * * *', ...) // 09:00 UTC daily (default)
cron.schedule('0 */6 * * *', ...) // every 6 hours
cron.schedule('0 9 * * 1', ...) // Mondays 09:00 UTC
```

Model

```
ANTHROPIC_MODEL=claude-sonnet-4-6
ANTHROPIC_MODEL=claude-haiku-4-5 # default
```

Agent commands

Command	What it does
<code>npm run dev:dry</code>	Daemon + cron; <code>DRY_RUN=true</code> skips IPFS and chain
<code>npm run dev</code>	Production daemon (needs env + deployed contract)
<code>RUN_NOW=true npm run dev</code>	Run one digest on startup, then keep scheduling
<code>npm run run-once</code>	One production digest, then exit
<code>npm run run-once:now</code>	Same, immediately (<code>RUN_ONCE=true RUN_NOW=true</code>)

`RUN_ONCE` and `DRY_RUN` cannot be combined (production-only).

Deep-dive articles

`generateDeepDive(topic)` in `agent/src/claude.ts` (1000–1500 words). Wire a small script to call it, then encrypt, pin, and `publishContent` with `contentType: 'Article'` .

Running the full demo

Prerequisites

- ☐ Vara testnet VARA on agent wallet and demo reader wallet
- ☐ `vara-wallet` CLI — `scripts/setup-agent.sh` creates `agent-wallet` and requests faucet
- ☐ Contract built and deployed — `bash scripts/deploy.sh`
- ☐ `agent/.env.local` and `frontend/.env.local` with matching `PROGRAM_ID`
- ☐ Pinata + Anthropic keys (for agent runs)
- ☐ Browser extension wallet (Talisman, SubWallet, or Polkadot{.js})

One-time setup

```
bash scripts/setup-agent.sh
bash scripts/deploy.sh
# Copy agent/.env.example → agent/.env.local and fill secrets
# Ensure frontend/.env.local has NEXT_PUBLIC_PROGRAM_ID and NEXT_PUBLIC_NETWORK
```

Patch IDL for wallet if needed:

```
bash scripts/fix-idl-for-wallet.sh agent/tella.idl
```

Seed content (pick one)

```
# Agent: one digest now, then exit
cd agent && npm run run-once:now

# Or dry-run only (no chain/IPFS)
cd agent && npm run dev:dry
# with RUN_NOW=true in another terminal session if you want immediate generation
```

Or publish manually at `/creator` with a CID you already pinned.

Start the frontend

```
cd frontend
npm run dev
# http://localhost:3000
```

Production deploy: `cd frontend && vercel --prod` (set env vars in Vercel).

Demo script (audience walkthrough)

A — Reader buys and reads

1. Open `/browse` — show cards (title, price, pass count).

2. Open a post — public title/description; gated body area.
3. **Connect Wallet** in the nav.
4. Click **Buy Access Pass — {n} VARA** — approve in the extension.
5. Wait for confirmation — UI shows “Access granted. Content unlocking...” then decrypts automatically (no separate “Read” button).
6. Show `/my-passes` for purchased content.

B — Creator revenue

1. Open `/creator` (nav label **Publish**).
2. **Check Balance** → pending VARA.
3. **Claim** → `Treasury/ClaimRevenue` .

C — Agent (optional terminal)

```
cd agent
npm run run-once:now
```

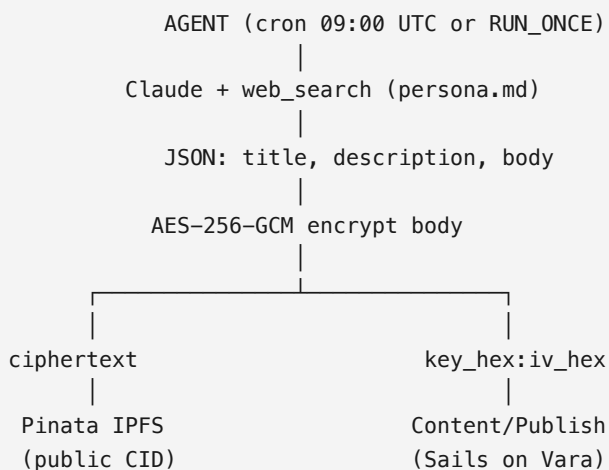
Example logs:

```
[agent] Starting digest generation...
[agent] Generated: "AI x Crypto Weekly #..."
[agent] Pinned to IPFS: bafy...
[agent] Published on-chain: content #2
```

For a long-running daemon:

```
cd agent && npm run dev
# Scheduled for 09:00 UTC; use RUN_NOW=true to fire once at startup
```

Architecture



|
ContentPublished event

READER (browser /post/[id])
|
/browse → Content/ListContent
connect wallet (extension)
Pass/MintPass + value
|
Content/RequestAccessKey (pass or creator)
|
/api/ipfs-proxy?cid=... → ciphertext
|
AES decrypt in browser → markdown render

CREATOR (/creator or agent wallet)
|
Treasury/GetPendingRevenue
Treasury/ClaimRevenue → VARA

Contract surface (Sails program ContentAgent)

Service	Methods used by Tella
Content	Publish , ListContent , GetContent , RequestAccessKey
Pass	MintPass , CheckAccess , GetUserPasses
Treasury	GetPendingRevenue , ClaimRevenue

Build artifacts: contracts/tella/target/wasm32-gear/release/tella.opt.wasm , tella.idl .